

Control del paso

C. Parés

Análisis Numérico Curso 2021-22



UNIVERSIDAD DE MÁLAGA

- 1 Introducción
- 2 Estimador del error de discretización local
- 3 Métodos RK encajados
- 4 Elección del nuevo paso
- 5 Implementación de los métodos RK
- 6 Implementación para sistemas

- Vamos a considerar métodos con *paso variable*:

$$y_{k+1} = y_k + h_k \Phi(t_k, y_k; h_k), \quad k = 0, 1, \dots$$

- **Idea:** Elegir h_k de manera que se tenga

$$\varepsilon_k \approx \varepsilon$$

donde

$$\varepsilon_k = y(t_{k+1}) - y(t_k) - h_k \Phi(t_k, y(t_k); h_k)$$

es el **error de discretización local** y ε una tolerancia de error que se da a priori.

- El error de discretización local no se puede calcular porque $y(t_k)$ e $y(t_{k+1})$ no son conocidos. Vamos a usar una **estimación del error**

$$\tilde{\varepsilon}_k \approx \varepsilon_k.$$

- 1 Introducción
- 2 Estimador del error de discretización local
- 3 Métodos RK encajados
- 4 Elección del nuevo paso
- 5 Implementación de los métodos RK
- 6 Implementación para sistemas

- **Idea:** Supongamos que el método es de orden p . Consideramos entonces un segundo método de orden mayor $p^* > p$ cuya función de incremento es $\Phi^*(t, y; h)$.
- Aproximamos $y(t_k)$ por y_k e $y(t_{k+1})$ por

$$y_{k+1}^* = y_k + h_k \Phi^*(t_k, y_k; h_k), \quad k = 0, 1, \dots$$

- Se tiene entonces

$$\begin{aligned} \varepsilon_k &= y(t_{k+1}) - y(t_k) - h_k \Phi(t_k, y(t_k); h_k) \\ &\approx y_{k+1}^* - y_k - h_k \Phi(t_k, y_k; h_k) \\ &= y_k + h_k \Phi^*(t_k, y_k; h_k) - y_k - h_k \Phi(t_k, y_k; h_k) \\ &= h_k (\Phi^*(t_k, y_k; h_k) - \Phi(t_k, y_k; h_k)) \\ &:= \tilde{\varepsilon}_k. \end{aligned}$$

- Veremos en clase que

$$\tilde{\varepsilon}_k = \varepsilon_k + O(h^{p+2}).$$

- 1 Introducción
- 2 Estimador del error de discretización local
- 3 Métodos RK encajados**
- 4 Elección del nuevo paso
- 5 Implementación de los métodos RK
- 6 Implementación para sistemas

- Se consideran dos métodos RK de orden p y $p+1$ con el mismo número de etapas, q , y tableros de la forma:

$$\begin{array}{c|ccc} c_1 & a_{1,1} & \dots & a_{1,q} \\ \vdots & \vdots & \ddots & \vdots \\ c_q & a_{q,1} & \dots & a_{q,q} \\ \hline & b_1 & \dots & b_q \end{array} \quad \begin{array}{c|ccc} c_1 & a_{1,1} & \dots & a_{1,q} \\ \vdots & \vdots & \ddots & \vdots \\ c_q & a_{q,1} & \dots & a_{q,q} \\ \hline & b_1^* & \dots & b_q^* \end{array}$$

es decir, con los mismos coeficientes $a_{i,j}$ y c_i , por lo que las etapas son idénticas para los dos métodos:

$$y_k^{(i)} = y_k + h_k \sum_{j=1}^q a_{i,j} f(t_k^{(j)}, y_k^{(j)}), \quad i = 1, \dots, q.$$

- Se usa el método de orden p para avanzar y el método de orden $p+1$ para estimar el error de discretización local:

$$y_{k+1} = y_k + h_k \sum_{i=1}^q b_i f(t_k^{(i)}, y_k^{(i)}),$$

$$\tilde{\varepsilon}_k = h_k (\Phi^*(t_k, y_k; h_k) - \Phi(t_k, y_k; h_k)) = h_k \sum_{i=1}^q (b_i^* - b_i) f(t_k^{(i)}, y_k^{(i)}).$$

- Se escribe un único tablero para el par RK $p(p+1)$:

c_1	$a_{1,1}$	$\dots$	$a_{1,q}$
$\vdots$	$\vdots$	$\ddots$	$\vdots$
c_q	$a_{q,1}$	$\dots$	$a_{q,q}$
	b_1	$\dots$	b_q
	b_1^*	$\dots$	b_q^*

- Ejemplo: RK2(3):

0	0	0	0
1/2	1/2	0	0
1	-1	2	0
	0	1	0
	1/6	2/3	1/6

- 1 Introducción
- 2 Estimador del error de discretización local
- 3 Métodos RK encajados
- 4 Elección del nuevo paso**
- 5 Implementación de los métodos RK
- 6 Implementación para sistemas

- En un método de orden p se tiene

$$\varepsilon_k = O(h^{p+1}).$$

- Si pasamos de un paso h_k a un paso h_{k+1} el paso se divide por h_k/h_{k+1} por lo que cabe esperar que el error de discretización local se divida por $(h_k/h_{k+1})^{p+1}$, es decir:

$$\frac{|\varepsilon_k|}{|\varepsilon_{k+1}|} \approx \left(\frac{h_k}{h_{k+1}} \right)^{p+1}.$$

es decir

$$|\varepsilon_{k+1}| \approx \left(\frac{h_{k+1}}{h_k} \right)^{p+1} |\varepsilon_k|.$$

- Si queremos que $|\varepsilon_{k+1}| \approx \varepsilon$, hacemos

$$\varepsilon \approx |\varepsilon_{k+1}| \approx \left(\frac{h_{k+1}}{h_k} \right)^{p+1} |\varepsilon_k| \approx \left(\frac{h_{k+1}}{h_k} \right)^{p+1} |\tilde{\varepsilon}_k|,$$

- Despejando

$$h_{k+1} = \left(\frac{\varepsilon}{|\tilde{\varepsilon}_k|} \right)^{\frac{1}{p+1}} h_k.$$

- En la práctica, por seguridad se hace:

$$h_{k+1} = \alpha \left(\frac{\varepsilon}{|\tilde{\varepsilon}_k|} \right)^{\frac{1}{p+1}} h_k.$$

donde $\alpha \in (0, 1)$.

- Si se usa un método $\text{RK}(p+1)$ explícito, para resolver el problema de Cauchy

$$\begin{cases} y' = f(t, y), & t \in [a, b], \\ y(0) = y_0 \end{cases}$$

el algoritmo se escribe así:

- Se eligen ε y h_0 .
- $t_0 = a$.
- Para $k = 0, 1, \dots$, mientras $t_k < b$:
 - $\tilde{h}_k = \min(h_k, b - t_k)$;
 - $t_k^{(i)} = t_k + c_i \tilde{h}_k, \quad i = 1, \dots, q$;
 - $y_k^{(i)} = y_k + \tilde{h}_k \sum_{j=1}^{i-1} a_{i,j} f(t_k^{(j)}, y_k^{(j)}), \quad i = 1, \dots, q$;
 - $y_{k+1} = y_k + \tilde{h}_k \sum_{i=1}^q b_i f(t_k^{(i)}, y_k^{(i)})$;
 - $\tilde{\varepsilon}_k = \tilde{h}_k \sum_{i=1}^q (b_i^* - b_i) f(t_k^{(i)}, y_k^{(i)})$;
 - $h_{k+1} = \alpha \left(\frac{\varepsilon}{|\tilde{\varepsilon}_k|} \right)^{\frac{1}{p+1}} \tilde{h}_k$;
 - $t_{k+1} = t_k + \tilde{h}_k$.

- 1 Introducción
- 2 Estimador del error de discretización local
- 3 Métodos RK encajados
- 4 Elección del nuevo paso
- 5 Implementación de los métodos RK**
- 6 Implementación para sistemas

- Un método RK puede escribirse de dos maneras:

$$\begin{aligned}t_k^{(i)} &= t_k + c_i h, \quad i = 1, \dots, q; \\ y_k^{(i)} &= y_k + h \sum_{j=1}^{i-1} a_{i,j} f(t_k^{(j)}, y_k^{(j)}), \quad i = 1, \dots, q; \\ y_{k+1} &= y_k + h \sum_{i=1}^q b_i f(t_k^{(i)}, y_k^{(i)}).\end{aligned}$$

o equivalentemente

$$\begin{aligned}t_k^{(i)} &= t_k + c_i h, \quad i = 1, \dots, q; \\ K_i &= f(t_k^{(i)}, y_k + h \sum_{j=1}^{i-1} a_{i,j} K_j), \quad i = 1, \dots, q; \\ y_{k+1} &= y_k + h \sum_{i=1}^q b_i K_i.\end{aligned}$$

- La segunda forma es más eficiente, porque solo se evalúa la f una vez en cada etapa $(t_k^{(i)}, y_k^{(i)})$.

- 1 Introducción
- 2 Estimador del error de discretización local
- 3 Métodos RK encajados
- 4 Elección del nuevo paso
- 5 Implementación de los métodos RK
- 6 Implementación para sistemas

- Queremos aplicar un método RK a un sistema:

$$\begin{cases} \vec{y}'(t) = \vec{f}(t, \vec{y}(t)) \\ \vec{y}(0) = \vec{y}_0, \end{cases}$$

con

$$\vec{y}(t) = \begin{bmatrix} y_1(t) \\ \vdots \\ y_L(t) \end{bmatrix}, \quad \vec{f}(t, \vec{y}) = \begin{bmatrix} f_1(t, y_1, \dots, y_L) \\ \vdots \\ f_L(t, y_1, \dots, y_L) \end{bmatrix}, \quad \vec{y}_0 = \begin{bmatrix} y_1^0 \\ \vdots \\ y_L^0 \end{bmatrix}.$$

- Un método RK para este problema se escribe así:

$$\vec{K}_i = \vec{f}\left(t_k^{(i)}, \vec{y}_k + h \sum_{j=1}^q a_{i,j} \vec{K}_j\right), \quad j = 1, \dots, q,$$

$$\vec{y}_{k+1} = \vec{y}_k + h \sum_{i=1}^q b_i \vec{K}_i.$$

- Obsérvese que las 'pendientes' $\vec{K}_i$ son vectores de L componentes:

$$\vec{K}_i = \begin{bmatrix} K_{1,i} \\ \vdots \\ K_{L,i} \end{bmatrix}, i = 1, \dots, q.$$

- En el programa, se almacenarán las pendientes en un array de L filas y q columnas:

$$\mathcal{K} = \left[\vec{K}_1 \mid \vdots \mid \vec{K}_q \right] = \begin{bmatrix} K_{1,1} & \dots & K_{1,q} \\ \vdots & \ddots & \vdots \\ K_{L,1} & \dots & K_{L,q} \end{bmatrix}$$

- Escrito en componentes, el sumatorio que aparece en la etapa i -ésima del método RK es:

$$\sum_{j=1}^q a_{i,j} \vec{K}_j = \begin{bmatrix} \sum_{j=1}^q a_{i,j} K_{1,j} \\ \vdots \\ \sum_{j=1}^q a_{i,j} K_{L,j} \end{bmatrix} = [a_{i,1}, \dots, a_{i,q}] \cdot \mathcal{K}^T.$$

- Un método explícito puede programarse así:

```
t = array([a])
y = zeros([len(y0),N+1])
y[:,0] = y0
K = zeros([len(y0),q])
for i in range(N):
    for i in range(q):
        K[:,i] = fun(t[k]+C[i]*h[k], y[:,k]+h[k]*dot(A[i,:],transpose(K)))
    y[:,k+1] = y[:,k] + h* dot(B, transpose(K))
```

- `dot(A,B)` hace el producto de una matrix $M \times N$ por una matrix $N \times L$.

- También el error de discretización local es un vector

$$\vec{\varepsilon}_k = \vec{y}(t_{k+1}) - \vec{y}(t_k) - h\vec{\Phi}(t_k, y(t_k); h)$$

y el estimador también

$$\tilde{\vec{\varepsilon}}_k = h \sum_{i=0}^q (b_i^* - b_i) \vec{K}_i.$$

- Lo que se desea ahora es

$$\|\vec{\varepsilon}_k\|_{\infty} \approx \varepsilon,$$

donde ε es la tolerancia de error y

$$\|\vec{y}\|_{\infty} = \max_{1 \leq j \leq q} |y_j|$$

es la norma infinito.

- El nuevo paso se calcula entonces con la fórmula:

$$h_{k+1} = \alpha \left(\frac{\varepsilon}{\|\tilde{\vec{\varepsilon}}_k\|_{\infty}} \right)^{\frac{1}{p+1}} \tilde{h}_k.$$

- La norma infinito de un array y se puede calcular así...

```
from numpy import linalg as LA  
LA.norm(y,inf))
```

- ... o así:

```
max(abs(y))
```